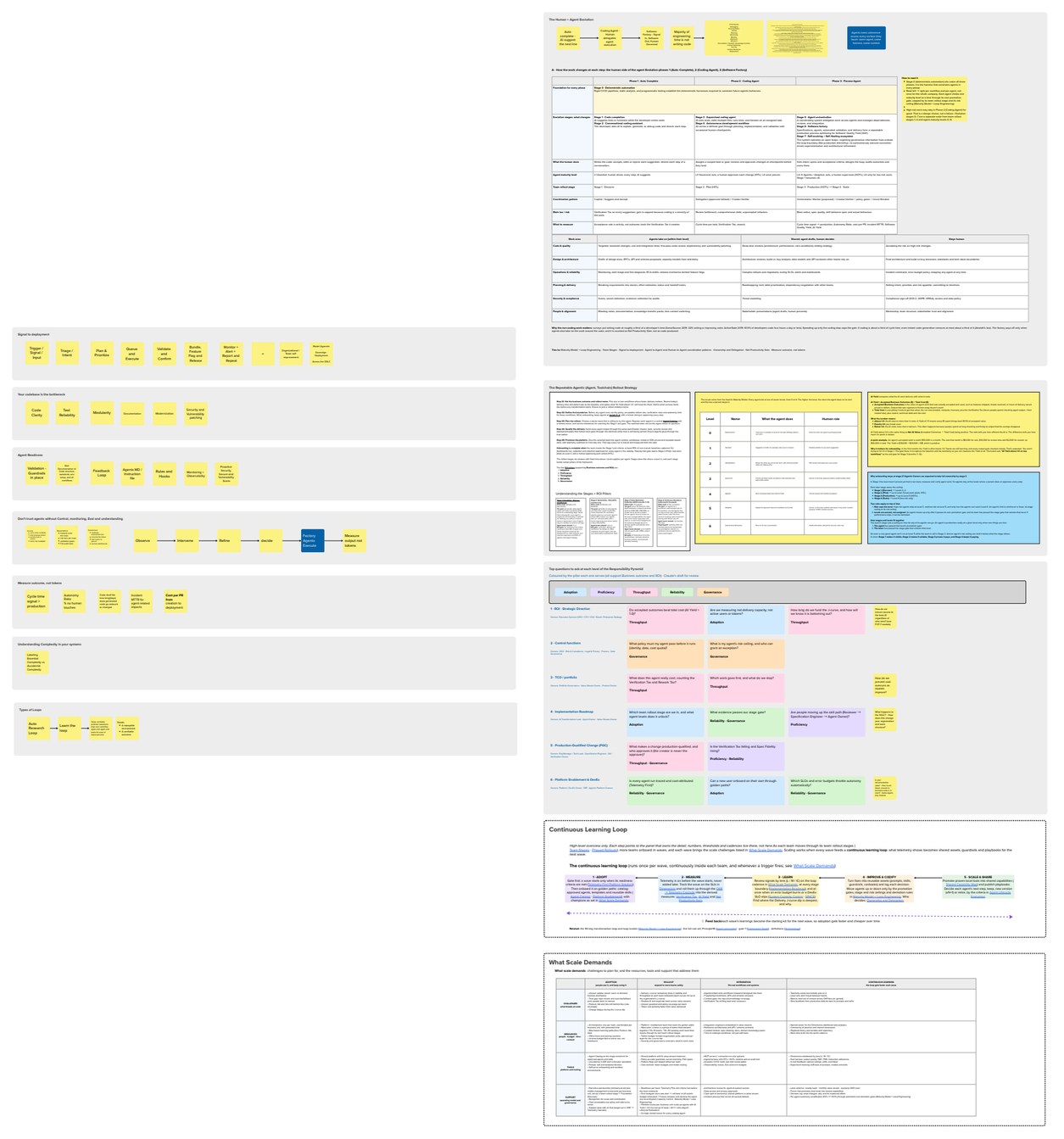
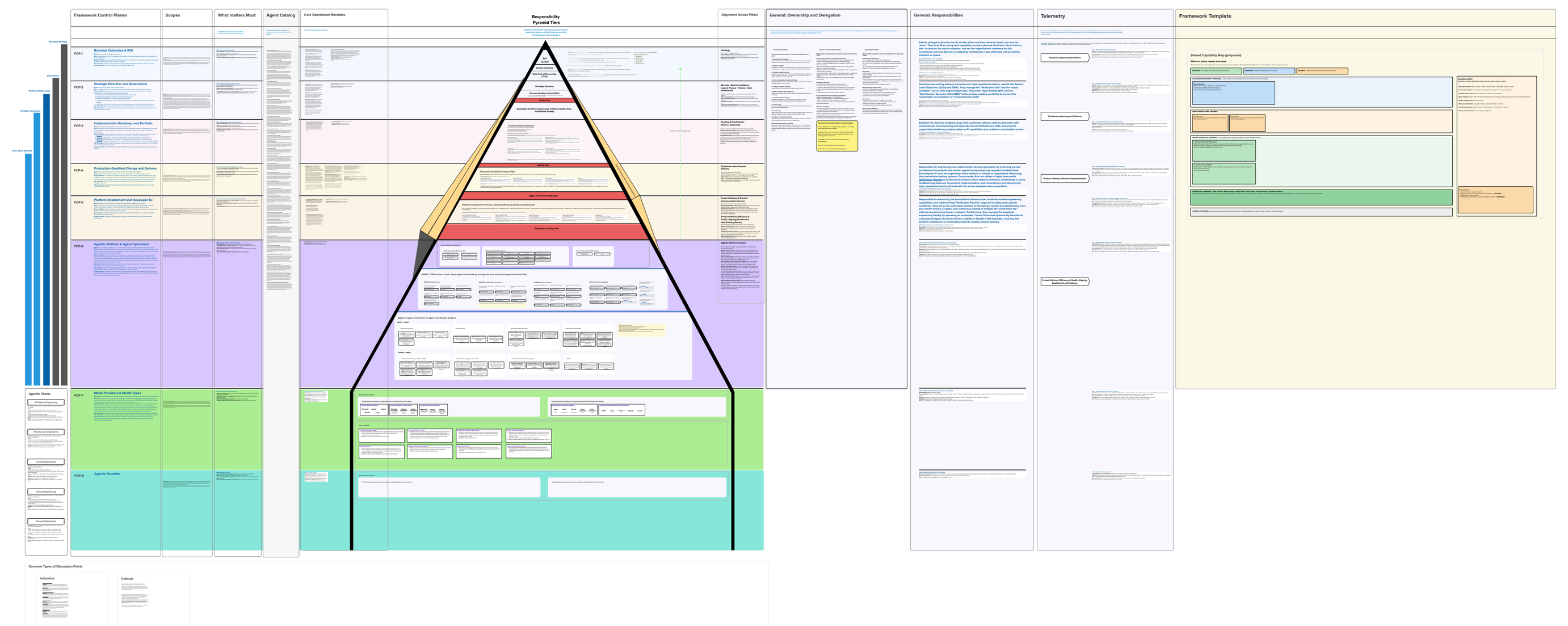
Framework DRAFT_1.1.0

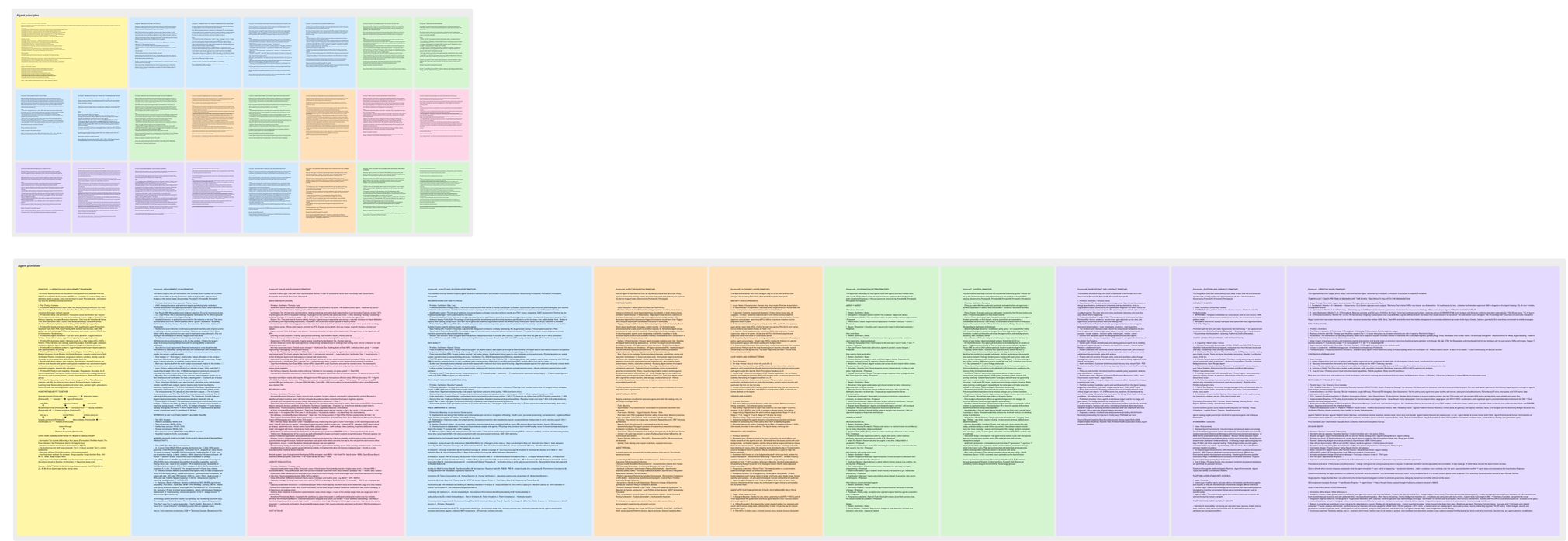
Agentic Perspective



The How:



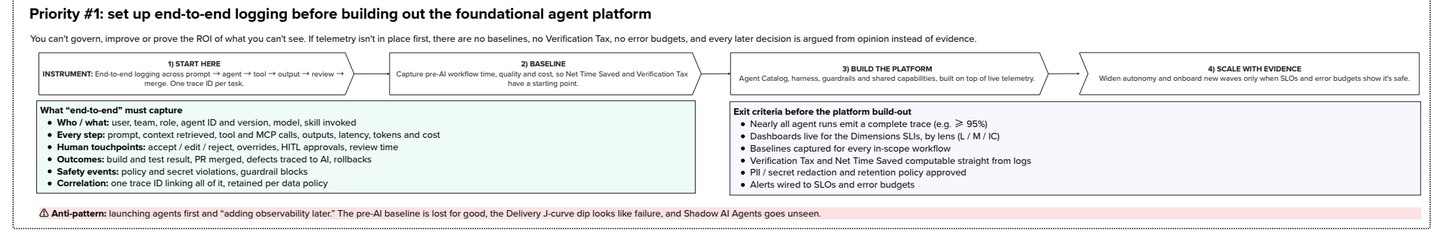
Agent principles + primitives



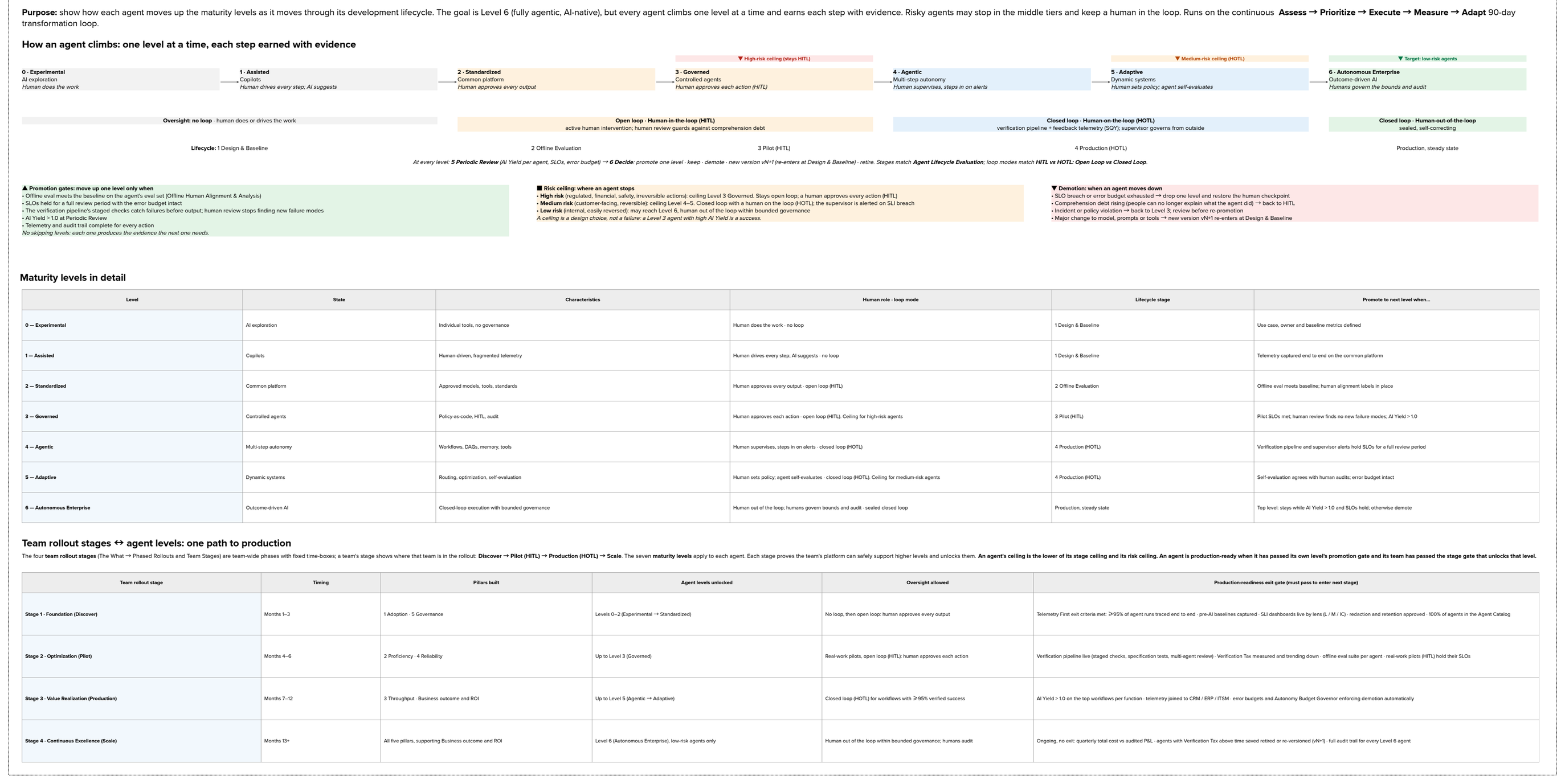
North Star: Goals, Expectations & Focus

Framework Goals

Telemetry First Platform Solution

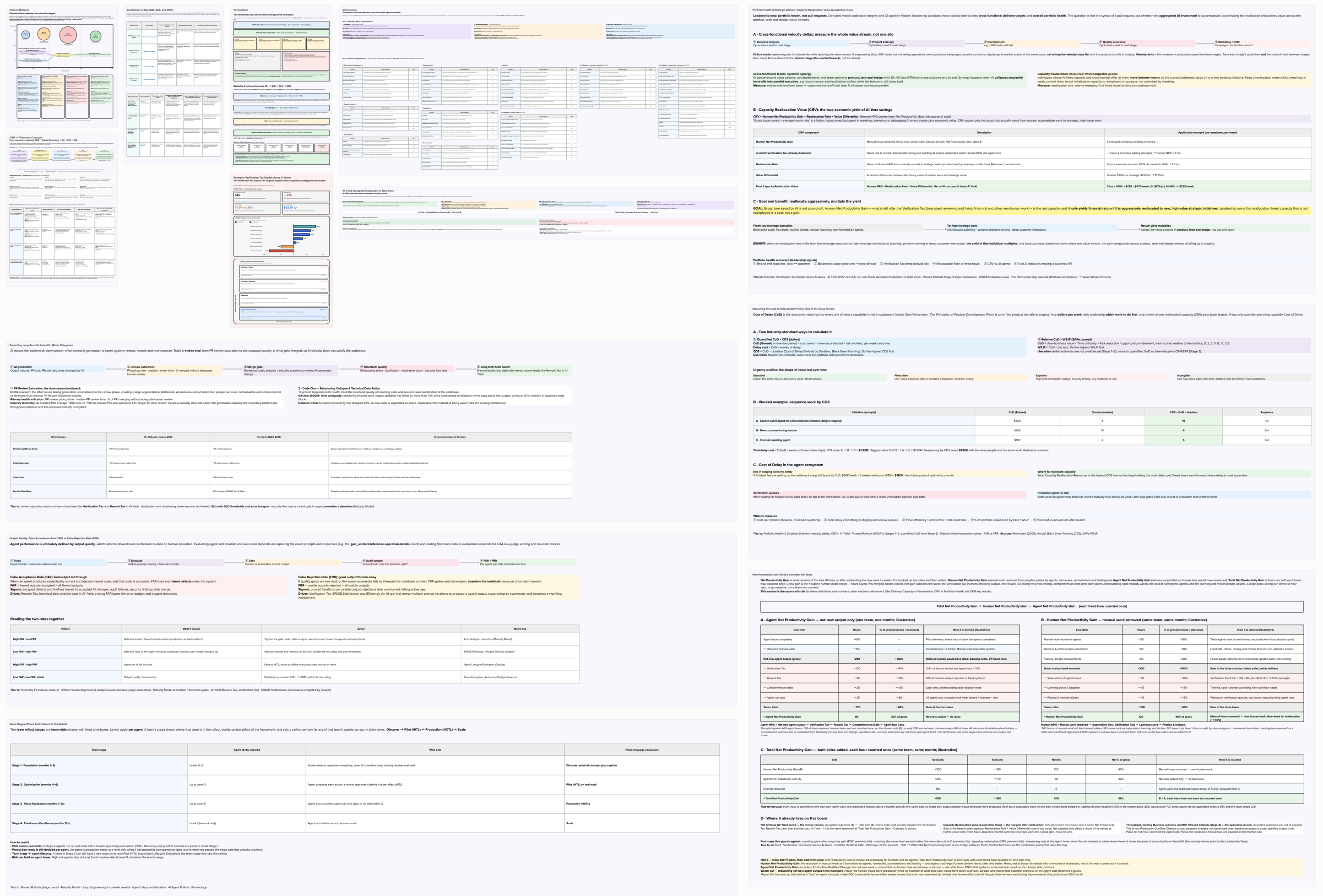


Maturity Model + Loop Engineering

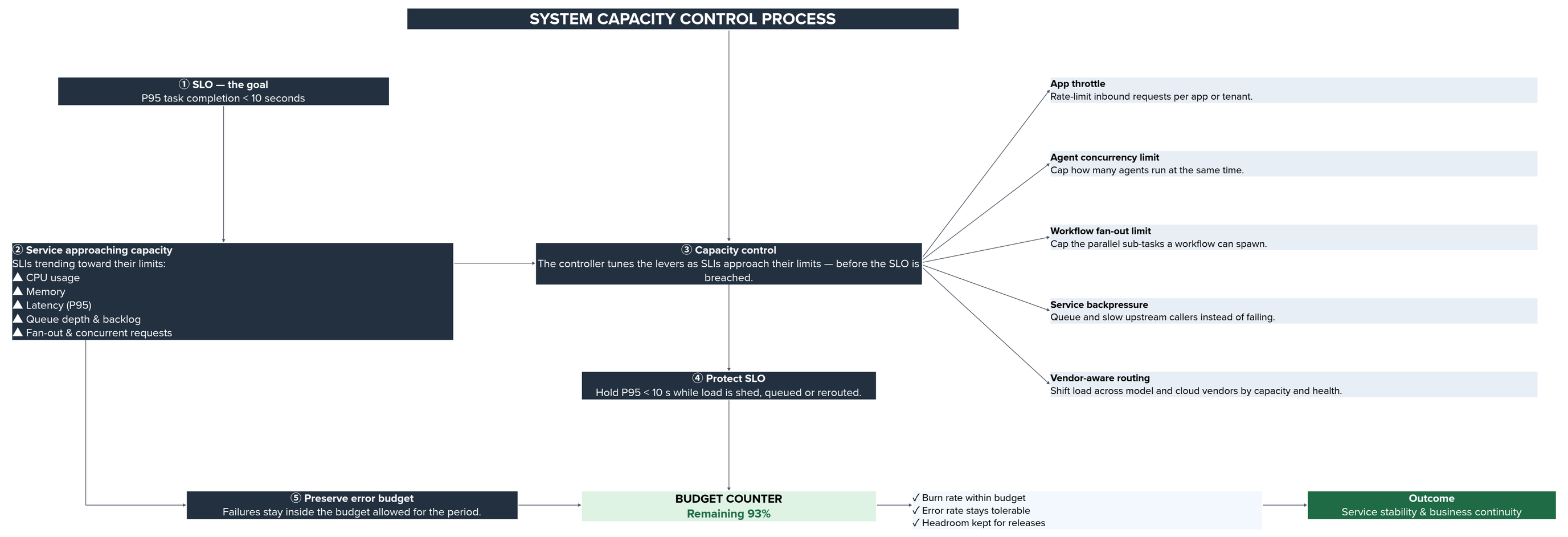


Terminology: Terms Used Across the Framework

The What: Measuring the frontier agent ecosystem



System Capacity Control: Protecting SLOs & Error Budgets

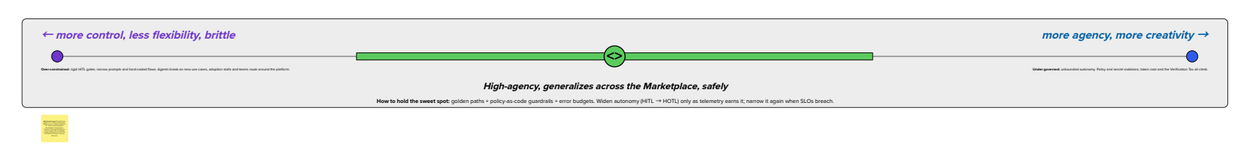

SYSTEM CAPACITY CONTROL PROCESS
① SLO — the goal
P95 task completion < 10 seconds
② Service approaching capacity
SLIs trending toward their limits:
▲ CPU usage
▲ Memory
▲ Latency (P95)
▲ Queue depth & backlog
▲ Fan-out & concurrent requests
③ Capacity control
The controller tunes the levers as SLIs approach their limits — before the SLO is breached.
④ Protect SLO
Hold P95 < 10 s while load is shed, queued or rerouted.
App throttle
Rate-limit inbound requests per app or tenant.
Agent concurrency limit
Cap how many agents run at the same time.
Workflow fan-out limit
Cap the parallel sub-tasks a workflow can spawn.
Service backpressure
Queue and slow upstream callers instead of failing.
Vendor-aware routing
Shift load across model and cloud vendors by capacity and health.
⑤ Preserve error budget
Failures stay inside the budget allowed for the period.
BUDGET COUNTER
Remaining 93%
✓ Burn rate within budget
✓ Error rate stays tolerable
✓ Headroom kept for releases
Outcome
Service stability & business continuity

Telemetry measures the SLI. The SLI determines whether the SLO is being achieved. The SLA defines the commitment. Controls protect the SLO/SLA. Error budgets determine how much failure is acceptable before corrective action is required.

Ties to: Breakdown of SLI, SLO, SLA and OKRs (The What) · error budgets + Autonomy Budget Governor (Stage 3 exit gate) · Telemetry First Platform Solution (North Star)

Agent Building

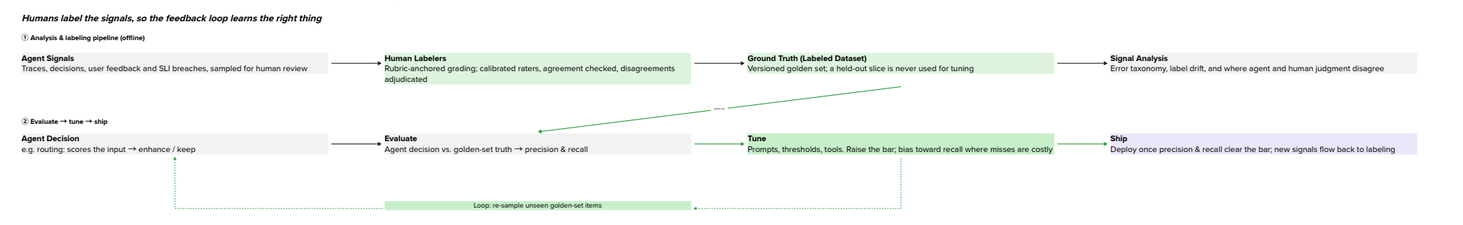
Agent Control: Deterministic vs Creative


← more control, less flexibility, brittle
more agency, more creativity →
High-agency, generalizes across the Marketplace, safely

How to hold the sweet spot: golden paths + policy-as-code guardrails + error budgets. Widen autonomy (HITL → HOTL) only as telemetry earns it; narrow it again when SLOs breach.

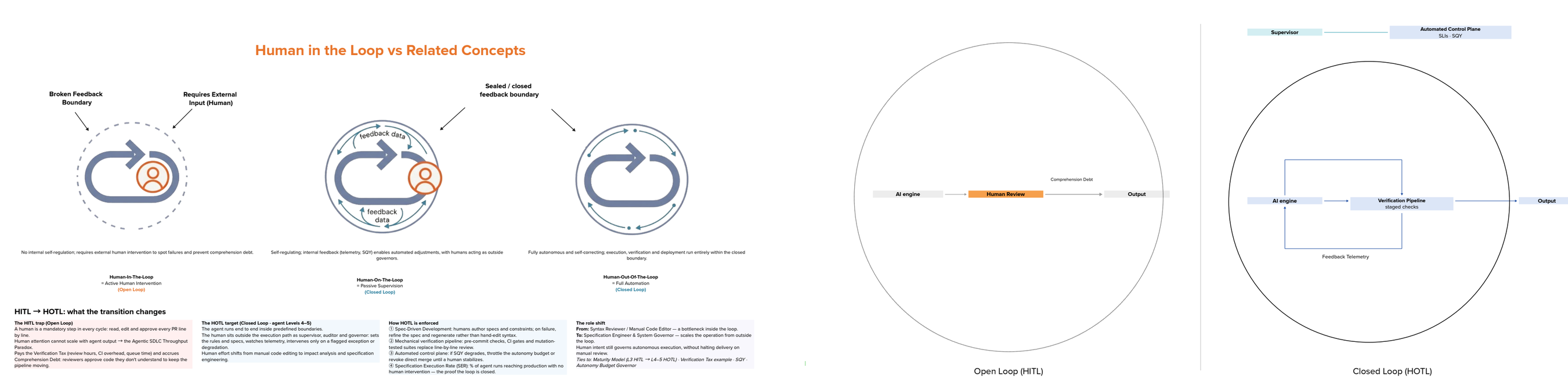
Offline Human Alignment & Analysis

Humans label the signals, so the feedback loop learns the right thing


① Analysis & labeling pipeline (offline)
Agent Signals
Traces, decisions, user feedback and SLI breaches, sampled for human review
Human Labelers
Rubric-anchored grading; calibrated raters, agreement checked, disagreements adjudicated
Ground Truth (Labeled Dataset)
Versioned golden set; a held-out slice is never used for tuning
Signal Analysis
Error taxonomy, label drift, and where agent and human judgment disagree
② Evaluate → tune → ship
Agent Decision
e.g. routing: scores the input → enhance / keep
Evaluate
Agent decision vs. golden-set truth → precision & recall
Tune
Prompts, thresholds, tools. Raise the bar; bias toward recall where misses are costly
Ship
Deploy once precision & recall clear the bar; new signals flow back to labeling
Loop: re-sample unseen golden-set items

Why it matters: the feedback loop is only as good as its labels. If signals are mislabeled, the agent is tuned toward the wrong target. Human alignment and label QA are part of agent building, not an afterthought.

HITL vs HOTL: Open Loop vs Closed Loop


Human in the Loop vs Related Concepts
Broken Feedback Boundary
Requires External Input (Human)
Sealed / closed feedback boundary
feedback data
No internal self-regulation; requires external human intervention to spot failures and prevent comprehension debt.
Self-regulating; internal feedback (telemetry, SQY) enables automated adjustments, with humans acting as outside governors.
Fully autonomous and self-correcting; execution, verification and deployment run entirely within the closed boundary.
Human-In-The-Loop
= Active Human Intervention
(Open Loop)
Human-On-The-Loop
= Passive Supervision
(Closed Loop)
Human-Out-Of-The-Loop
= Full Automation
(Closed Loop)
AI engine
Human Review
Comprehension Debt
Output
Open Loop (HITL)
Supervisor
Automated Control Plane
SLIs · SQY
AI engine
Verification Pipeline
staged checks
Feedback Telemetry
Output
Closed Loop (HOTL)

HITL → HOTL: what the transition changes

The HITL trap (Open Loop)
A human is a mandatory step in every cycle: read, edit and approve every PR line by line.
Human attention cannot scale with agent output → the Agentic SDLC Throughput Paradox.
Pays the Verification Tax (review hours, CI overhead, queue time) and accrues Comprehension Debt: reviewers approve code they don't understand to keep the pipeline moving.

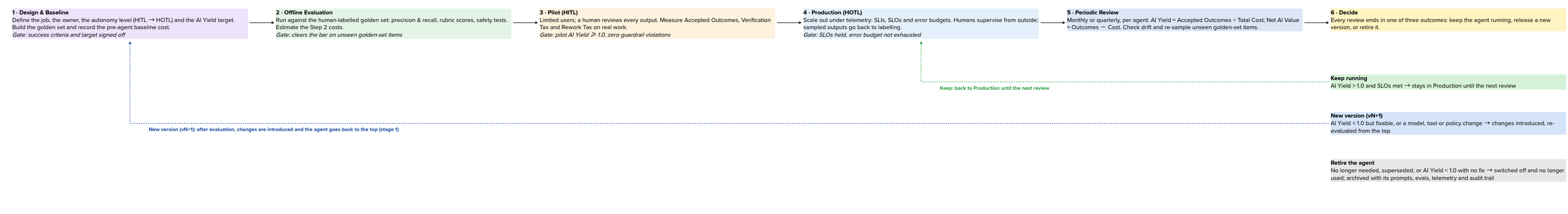
The HOTL target (Closed Loop · agent Levels 4–5)
The agent runs end to end inside predefined boundaries.
The human sits outside the execution path as supervisor, auditor and governor: sets the rules and specs, watches telemetry, intervenes only on a flagged exception or degradation.
Human effort shifts from manual code editing to impact analysis and specification engineering.

How HOTL is enforced
① Spec-Driven Development: humans author specs and constraints; on failure, refine the spec and regenerate rather than hand-edit syntax.
② Mechanical verification pipeline: pre-commit checks, CI gates and mutation-tested suites replace line-by-line review.
③ Automated control plane: if SQY degrades, throttle the autonomy budget or revoke direct merge until a human stabilizes.
④ Specification Execution Rate (SER): % of agent runs reaching production with no human intervention — the proof the loop is closed.

The role shift
From: Syntax Reviewer / Manual Code Editor — a bottleneck inside the loop.
To: Specification Engineer & System Governor — scales the operation from outside the loop.
Human intent still governs autonomous execution, without halting delivery on manual review.
Ties to: Maturity Model (L3 HITL → L4–5 HOTL) · Verification Tax example · SQY · Autonomy Budget Governor

Agent Lifecycle Evaluation

Every agent is evaluated on AI Yield at every stage of its life, not just at launch


1 · Design & Baseline
Define the job, the owner, the autonomy level (HITL → HOTL) and the AI Yield target. Build the golden set and record the pre-agent baseline cost.
Gate: success criteria and target signed off
2 · Offline Evaluation
Run against the human-labelled golden set: precision & recall, rubric scores, safety tests. Estimate the Step 2 costs.
Gate: clears the bar on unseen golden-set items
3 · Pilot (HITL)
Limited users; a human reviews every output. Measure Accepted Outcomes, Verification Tax and Rework Tax on real work.
Gate: pilot AI Yield ≥ 1.0, zero guardrail violations
4 · Production (HOTL)
Scale out under telemetry: SLIs, SLOs and error budgets. Humans supervise from outside; sampled outputs go back to labelling.
Gate: SLOs held, error budget not exhausted
5 · Periodic Review
Monthly or quarterly, per agent: AI Yield = Accepted Outcomes ÷ Total Cost; Net AI Value = Outcomes − Cost. Check drift and re-sample unseen golden-set items.
6 · Decide
Every review ends in one of three outcomes: keep the agent running, release a new version, or retire it.
Keep running
AI Yield > 1.0 and SLOs met → stays in Production until the next review
Keep: back to Production until the next review
New version (vN+1)
AI Yield < 1.0 but fixable, or a model, tool or policy change → changes introduced, re-evaluated from the top
New version (vN+1): after evaluation, changes are introduced and the agent goes back to the top (stage 1)
Retire the agent
No longer needed, superseded, or AI Yield < 1.0 with no fix → switched off and no longer used; archived with its prompts, evals, telemetry and audit trail

Measured at every stage (from AI Yield): Step 1 Accepted Business Outcomes ($) · Step 2 Total Cost (operational, Verification Tax, Rework Tax, technical debt, risk) · Step 3 AI Yield = Step 1 ÷ Step 2 · Step 4 Net AI Value = Step 1 − Step 2

Rules: each new version starts again at stage 1 and earns its autonomy back · AI Yield is tracked per version, so versions can be compared · retired agents are archived, not deleted

SPACE Framework: Measuring the Individual Contributor & Developer Experience

Level 2 of the operating model. Agent telemetry must be read in the human–computer interaction layer: for any individual contributor (engineer, marketer, analyst) the goal is task execution, flow state and less cognitive load. Measuring raw generation volume (lines of code, words written) rewards bloated repositories and ignores the actual developer experience.